模型压缩横纵分析报告

> 从 8-bit 到 4-bit 再到 FP4，从 GPTQ 到 AWQ 到 GGUF，大模型如何在精度、大小和速度之间走出"既要又要还要"的路。

一、一句话定义

模型压缩（Model Compression）是**在尽可能保持模型质量的前提下，减小模型体积、降低推理计算量的技术集合**。它的核心矛盾在于：模型越大、越精确，但部署成本和延迟也越高——压缩是在"精度损失"和"效率提升"之间找最优平衡点。

二、技术背景

核心矛盾：模型越强，部署越难

从 2018 年的 BERT-Base（110M 参数, 440MB）到 2026 年的 GPT-5.5（数万亿参数），6 年间模型规模增长了约 10000 倍。如果不做压缩：

- **显存**：一个 70B FP16 模型需 140GB 显存——4 块 A100 80G 才跑得动。每次推理成本约 $0.5-1。
- **延迟**：KV Cache 随序列长度二次增长，128K 上下文时单次生成可能需要几十秒。
- **吞吐**：如果不压缩，高吞吐服务的 GPU 成本线性增长。

模型压缩的所有工作都围绕一个核心洞察：**模型参数中存储了大量冗余信息——不是每个比特都对输出质量同等重要。**量化和剪枝利用这个冗余，蒸馏则在参数量级上直接缩减。

> 前置知识请参考独立文档 《模型压缩_前置知识》，涵盖量化、蒸馏、剪枝等基础概念的通俗讲解。

三、纵向分析：从边缘设备必需品到 LLM 标配

3.1 深度学习时代：压缩技术的起源（2015-2021）

模型压缩的三条技术线在深度学习时代就已经成熟，但在 LLM 时代之前，它们主要服务于边缘设备（手机、IoT）上的模型部署。

蒸馏（2015）：Geoffrey Hinton 在 "Distilling the Knowledge in a Neural Network"（NIPS 2014 Workshop）中正式提出知识蒸馏——训练一个"学生"网络去匹配"老师"网络的软目标（soft targets），而非直接学习硬标签。Hinton 的直觉是：老师输出的概率分布（如"这张图 80% 像猫，15% 像狗，5% 像兔子"）比硬标签（"猫"）包含了更多信息。

在 2015–2021 年间，蒸馏被广泛用于图像分类和语音识别，但核心应用场景是**模型大小受限的移动端部署**——从 ResNet-152 蒸馏到 MobileNet，从 BERT-Large 蒸馏到 TinyBERT。

量化（2016–2021）：量化的历史比蒸馏更长。早期的 INT8 量化（TensorRT, 2017）主要用于 NVIDIA GPU 推理加速。2020 年，Google 的 BERT INT8 量化和 Intel 的 INC（Neural Compressor）证明了 NLP 模型可以被压缩到 8-bit 而精度损失可忽略。

剪枝（2019）：Frankle & Carbin 在 "The Lottery Ticket Hypothesis"（ICLR 2019）中提出：大型神经网络的子网络（winning tickets）在独立训练时可以达到跟原网络相当的精度。这个假设引发了对结构化剪枝的大量研究。

这个阶段的本质是：**三套技术的核心原理在深度学习时代已经奠定，但 LLM 的到来将它们的"紧迫性"从"不错"提升到了"必须"。**

3.2 GPTQ 与量化转折点：让 LLM 部署变得可行（2022–2023）

2022–2023 年是 LLM 量化的关键转折期。在 GPT-3（2020）和 ChatGPT（2022.11）大火之后，所有人都想要部署自己的 LLM——但 175B 参数模型的部署成本远超大多数团队的承受能力。

GPTQ（2022.10, arXiv:2210.17323） 是 Frantar 等人的突破性工作。它将 Optimal Brain Quantizer（OBQ）改造为不适配 LLM 的海量参数场景：用 128 个校准样本，在 4 小时内将 OPT-175B 从 FP16 量化为 INT4，perplexity 损失不到 0.5。Hugging Face 上第一个广泛使用的 LLM 量化方法。

GPTQ 的意义在于：**它第一次证明了 LLM 可以被量化为 4-bit 而几乎不损失质量。** 它的二阶优化方法（用 Hessian 矩阵评估每个权重的重要性）虽然计算复杂，但在质量上设定了基准。

SmoothQuant（2022.11, arXiv:2211.10438） 解决了另一个问题：LLM 的激活值中存在极端异常值（比正常值大 100x+），导致直接 INT8 量化激活值时精度崩溃。SmoothQuant 将激活值的"量化难度"平滑迁移到权重上，实现了 W8A8（权重和激活都是 8-bit）的 LLM 推理加速。

AWQ（2023.06, arXiv:2306.00978） 来自 MIT 韩松团队，代表了另一个方向：与其对所有参数一视同仁地量化，不如观察激活值的分布，找到"重要"的 1% 权重通道做更高精度保护，其余大胆量化。AWQ 的理论巧妙——不需要校准集上的反向传播，只需统计激活幅度。

AWQ 在 2023–2024 年的工程能力快速超越 GPTQ。在同等位宽下，AWQ 的推理速度通常比 GPTQ 快 10–20%，质量接近或略优。到 2026 年，AWQ 成为 vLLM 和 TensorRT-LLM 的首选量化格式。

GGUF 与 llama.cpp（2023.03） 是社区工程力量的爆发。ggerganov 的 llama.cpp 项目将"在 CPU 上运行 LLaMA"变成了一个周末就能做的事。GGUF 作为其原生格式，支持从 Q2_K 到 Q8_0 的全系列量化级别，无需校准数据，直接内存映射加载。

GGUF 的统治地位不是技术上的（它的质量通常略低于 GPTQ/AWQ），而是工程上的——配合 LM Studio、Ollama 等前端工具，GGUF 构成了 2026 年最完整的本地 LLM 运行生态。对于"不想折腾、让模型跑起来就行"的用户，GGUF 是唯一选择。

SpQR（2023.06, arXiv:2306.03078） 走了一条更极致的路：找出"异常权重"并用更高精度单独存储，其余大胆量化到 3–4-bit。在 1% 左右的精度损失下将模型压缩到 4-bit 以下，但实现复杂度和运行时效率限制了它的大规模工程落

地。

这个阶段的本质是：**学术界和工程界同时意识到——LLM 的冗余比想象中大得多，4-bit 是最佳实用点。**

3.3 降低比特率：向 2-bit 和 1-bit 冲锋（2024-2025）

在 4-bit 成为标配之后，研究者的目标自然地转向了更低比特。

AQLM（2024.01, arXiv:2401.06118） 是 Yandex 和 IST Austria 的合作成果，目标是在 2-bit 级别做到可接受的质量。核心创新是残差量化——每个权重块用一个码本（codebook）表示，多级残差逐步缩小误差。在 2-bit 级别，AQLM 的 perplexity 显著优于 GPTQ 和 AWQ。

QuIP#（2024.02, arXiv:2402.04396, ICML 2024） 从信息论角度提出了一种新的量化框架。核心思想是"潜在处理"（lattice trellis coding）——用结构化的码本替代逐元素量化，在 2-bit 下达到当时最优质量。

FP8 硬件原生支持（2024-2025）：NVIDIA Hopper（H100/H200）原生支持 FP8 计算。FP8 推理速度约为 FP16 的 2 倍，TensorRT-LLM 和 vLLM 迅速跟进。

2025-2026 年，FP8 混合精度训练成为行业标准——BF16 训练正在被 FP8 取代。**推理的 FP8 标准也在形成**：权重用 E4M3（更高精度），激活用 E5M2（更大范围）。

但这个阶段的"2-bit 突破"更多是学术上的——在大规模工程部署中，2-bit 量化仍受限于：

- 特殊内核的缺乏（不像 4-bit 有 Marlin 等成熟内核）
- 异常值敏感任务（代码、数学）上的质量不可控
- 硬件加速有限

3.4 蒸馏在 LLM 时代的复兴（2024-2026）

蒸馏在 LLM 时代的路线跟传统蒸馏有本质不同。

DeepSeek-R1 蒸馏（2025.01） 是 LLM 蒸馏的里程碑。DeepSeek 用 R1 671B（MoE，实际激活 37B）生成了大量包含推理链的数据，然后用这些数据微调 Qwen 和 LLaMA 的小模型。

结果惊人：Distill-Qwen-32B 在 MATH 上达到 83.8%，接近 GPT-4 的 84.1%。7B 模型在数学推理上超越 GPT-4o mini。这不是传统的"soft label 蒸馏"，而是**推理方式蒸馏**——学生模型学到的不是老师的最终答案，而是老师的思考过程。

Microsoft Phi 系列（2023-2025） 走了另一条路：不是从单一 teacher 蒸馏，而是用大量合成数据训练小模型。Phi-1（1.3B, 2023.06）用"教科书质量"的合成数据训练，就达到了参数量 10x 模型的水平。Phi-4（14B, 2024.12）在 MMLU 上接近 84%，在数学推理上接近 Llama-3 70B。

Phi 系列证明了：**数据质量 > 参数量。** 这不是严格意义上的"蒸馏"（因为没有单个 teacher），但本质上是在利用大模型的知识来生成训练数据，是"数据层面的蒸馏"。

蒸馏在 LLM 时代的现实影响超过了量化：**70B 模型蒸馏到 7B + 4-bit 量化 → 最终模型大小从 140GB 降到 4GB，在 MacBook 上就能跑，特定领域能力接近原版的 80-90%。**

3.5 剪枝：最不成功的一条路（2023-2026）

剪枝在 LLM 时代的发展比量化和蒸馏困难得多。

SparseGPT（2023.01, arXiv:2301.00774） 是剪枝领域最有影响力的工作。Frantar 和 Alistarh（也是 GPTQ 的作者团队）提出了一种一步式剪枝方法，不需要训练或微调，就能移除 OPT-175B 中 50% 的参数——这是一个令人惊讶的发现。

Wanda（2023.06, arXiv:2306.11695） 进一步简化：只考虑权重和激活值的乘积，不需要 Hessian 矩阵计算。速度比 SparseGPT 快得多，质量接近。

问题在于：**剪枝后的稀疏模型在标准硬件上几乎没有速度提升**——通用的 GPU 矩阵乘法内核不支持稀疏计算。要真正从剪枝受益，需要 N:M 稀疏（NVIDIA Ampere 及更新架构支持 2:4 稀疏）或专用稀疏计算内核——后者在 2026 年仍未普及。

量化同时降低存储和计算需求，蒸馏降低模型参数量级——两种方法都在标准硬件上直接受益。剪枝只降低存储，不降低计算（除非特殊硬件），这是它在 LLM 时代未能成为主流的原因。

3.6 聚合趋势：FP8 训练 + 4-bit 推理成标准（2025-2026）

2025-2026 年，模型压缩的格局趋于稳定：

场景	推荐方案	模型大小示例（70B）
训练	FP8 混合精度	—
服务端高吞吐	AWQ 4-bit + vLLM / TensorRT-LLM	~40GB
端侧/本地	GGUF Q4_K_M + llama.cpp / LM Studio	~40GB
Apple Silicon	MLX 4-bit	~40GB
极致压缩	Q3_K_M / IQ3_XXS	~25GB
领域特化	蒸馏（7B）+ 量化 4-bit	~4GB

Blackwell FP4（2025-2026） 是下一个关键演变。NVIDIA B200 原生支持 FP4，TensorRT-LLM 已适配。NVFP4 KV Cache 可将 KV Cache 显存降低 4 倍——这对于长上下文推理场景是巨大的突破。

四、横向分析：竞争图谱

4.1 场景判断

模型压缩是充分竞争的市场（场景 C），从量化方法、推理框架到蒸馏方案，每个子方向都有多个成熟选择。

4.2 量化方法对比

方法	发布时间	核心原理	位宽	质量	推理速度	需要校准	GitHub 生态
GPTQ	2022.10	OBQ + Hessian 二阶优化	2/3/4/8-bit	高	中等	是 (128 samples)	6k+ stars
AWQ	2023.06	激活值感知，保护 1% 重要通道	4-bit	更高	**最快**	是 (128 samples)	5k+ stars
SmoothQuant	2022.11	逐通道平滑，激活值 INT8	8-bit	高	快	否	通用组件
GGUF	2023.03	逐块 MSE 最小化	Q2-Q8, IQ	中-高	中等	**不需要**	75k+ (llama.cpp)
NF4	2023	正态分布适配 4-bit	4-bit	高	中等	否	7k+ (bitsandbytes)
AQLM	2024.01	残差码本量化	**2-bit**	中	慢	是	2k+
QuIP#	2024.02	Lattice trellis coding	**2-bit**	中	慢	是	1k+

关键数据对比（Llama-2 7B wiki perplexity 越低越好）：

方法	FP16	Q8_0	Q4_K_M	Q4 (AWQ)	Q4 (GPTQ)	Q3_K_M	Q2_K
PPL	5.12	5.13	5.24	~5.20	~5.25	5.56	6.22

GGUF Q4_K_M 的 perplexity 惩罚约 0.12（2.3%），AWQ 约 0.08（1.6%）——对于大多数场景，这个差异人类不可感知。

4.3 推理框架量化支持对比

框架	支持格式	最佳场景	发布时间	GitHub Stars
llama.cpp	GGUF (Q2-Q8, IQ)	本地/CPU/端侧	2023.03	~75k
vLLM	AWQ, GPTQ, FP8, INT4, GGUF	**通用高吞吐**	2023.06	~78k
TensorRT-LLM	FP4, FP8, INT8, INT4, AWQ, GPTQ	NVIDIA 生产环境	2023.10	~15k
SGLang	AWQ, GPTQ, FP8	结构化生成	2024.01	~15k
ExLlamaV2	GPTQ, EXL2	**单 GPU 极速推理**	2023.07	~6k
MLX	4-bit, 8-bit	Apple Silicon	2023.12	~20k
bitsandbytes	NF4, FP4, INT8	PyTorch 训练/推理	2022.08	~7k

选型逻辑：

- **vLLM**：量化支持最广泛（从 AWQ 到 GGUF 到 bitsandbytes），配合 Marlin 内核在 Ada/Hopper GPU 上吞吐最高。2026 年的通用首选。
- **llama.cpp**：本地运行的绝对王者。配合 LM Studio 和 Ollama，"让模型在你的笔记本上跑"的唯一实际方案。
- **TensorRT-LLM**：如果你在 NVIDIA 硬件上做大规模生产部署，TensorRT-LLM 的 NVIDIA 内核深度优化无可替代。Blackwell FP4 的唯一支持框架。
- **ExLlamaV2**：单 GPU 场景的"隐藏王者"——在 RTX 4090 上跑 70B Q4 的速度可能超过 vLLM。

4.4 蒸馏方案对比

项目	方法	学生模型	学生 MMLU	学生 MATH	教师模型
DeepSeek-R1	推理链 SFT	Qwen-32B → Distill	~72%	~84%	R1 671B
DeepSeek-R1	推理链 SFT	Qwen-7B → Distill	~67%	~73%	R1 671B
Phi-4	合成数据训练	14B 从头训练	~84%	~82%	GPT-4 数据
Phi-3	合成数据 + 精筛	3.8B/7B/14B	~69%	—	—
Gemma 2	Logit-level 蒸馏	9B/27B	~75%	~58%	Gemini

DeepSeek-R1 的蒸馏路线被证明是目前最有效的方式——不是因为方法多高明，而是因为**teacher 模型的推理数据质量极高**。这与 Phi 系列的"合成数据"路线形成了有趣的对照：前者强调"怎么生成数据"（用最好的模型），后者强调"怎么筛选数据"（教科书质量）。

4.5 量化 vs 蒸馏 vs 剪枝——终极对比

维度	量化	蒸馏	剪枝
部署普遍性	最高（默认配置）	高（特定领域）	低
压缩比	4x (4-bit) / 2x (8-bit)	10x+ (70B→7B)	2x (50% 稀疏)
精度损失	极小 (Q4) / 可忽略 (Q8)	领域相关	中等-大
推理加速	明显（小模型/低比特内核）	明显（模型更小）	几乎无（硬件不支持）
硬件依赖	低（通用 CPU/GPU）	无	高（NVIDIA 2:4 稀疏）
流程复杂度	低（即量即用）	高（需要 teacher + SFT）	中
2026 年推荐	✅ 必做	✅ 领域特化首选	❌ 等硬件支持

4.6 社区生态量级对比

本地运行（GGUF + llama.cpp）是 2026 年模型压缩最大的社区生态。75k+ stars 和数千个 GGUF 模型库让它成为个人开发者的首选。

Hugging Face 生态中，GPTQ 模型数量仍然最多（最大基数），但 AWQ 的增长速度更快（被 vLLM 和 TensorRT-LLM 同时支持的复合效应）。

一个有趣的竞争格局：量化框架的竞争不是在"质量"上——几种主流方法的实际质量差异极小，人类几乎无法区分。竞争的核心是：

1. **生态兼容性**：vLLM 支持越多格式 → 该格式的模型越多 → 开发者选择该格式
2. **工具链完善度**：GGUF 的统治来自 LM Studio + Ollama + text-gen-webui 的完整体验
3. **推理速度**：AWQ → Marlin 内核 → vLLM → 高速推理，形成正向循环

五、横纵交汇洞察

5.1 量化的"超预期"成功：为什么它比理论上更好?

一个反直觉的事实：4-bit 量化理论上应该丢失大量信息（从 16-bit 降到 4-bit，精度损失 75%），但实际质量损失极小（perplexity 增加不到 3%）——这个"为什么"至今没有圆满的理论解释。

最被接受的假说是：**LLM 的参数空间存在巨大的冗余。** 训练过程中的随机梯度下降本身引入了大量噪声，模型学会在噪声中保持鲁棒性——量化的"冲击"只是另一种噪声。模型对噪声的宽容度远高于理论估计。

这个假说的启发性在于：**量化的"甜点"不是由理论精度限制决定的，而是由模型在训练中接触到的噪声水平决定的。** 如果这个假说成立，更大、训练更充分的模型，对量化的容忍度反而更高——而实践恰好证明了这一点（70B 量化到 4-bit 的损失比 7B 小得多）。

5.2 蒸馏的"非主流"崛起

纵向历史中最有趣的变化是蒸馏的路线变化。2015 年 Hinton 定义的"soft target"蒸馏在 LLM 时代被事实上的"data-level"蒸馏（合成数据 + SFT）取代——不是因为学术界不认可 Hinton 的框架，而是因为在 LLM 规模下：

- Soft target 蒸馏需要 logits 级别的访问——闭源 API 不可能提供
- Data-level 蒸馏只需要模型的输出——任何 API 都能获取
- Data-level 的数据还可以被筛选、增强、去重——质量可能超过原始 teacher

DeepSeek-R1 的推理链蒸馏本质上是**一种更高效的模型压缩**：不是压缩参数，而是压缩"能力"——student 模型不需要 671B 参数去理解数学，它只需要记住 teacher 的解题路径。这个思路可能比量化更接近模型压缩的本质：**压缩的不是数字，是知识。**

5.3 "4-bit 是否够用"的争论已经平息

2023-2024 年，社区反复争论 "4-bit 到底够不够用"。到 2026 年，这个争论基本平息了——不是因为 4-bit 被证明无损，而是因为：

1. Q4_K_M 和 AWQ 在实际使用中，用户无法可靠地区分与 FP16 的差异
2. 对于大多数消费级应用（聊天、摘要、翻译），微小的质量损失毫无影响
3. 对于需要极致质量的场景（代码生成、数学推理），保留 Q5_K_M 或 Q8 作为选项

争论平息的模式跟"RAG is Dead"争论如出一辙——不是一方赢了，而是行业认识到"不同场景需要不同配置"。4-bit 不是"最好"的，但它是"大多数情况下最合理"的。

5.4 "先剪枝再量化"的流水线为什么没成主流

理论上最优的压缩流程应该是：先剪枝移除冗余参数，再量化降低剩余参数的精度。但实践中，这条流水线在 LLM 上几乎没有被大规模使用。

原因在横纵对比中已经清楚：

- 剪枝的收益在标准硬件上不兑现（没有速度提升）
- 量化本身已经能做到 4x 压缩，边际收益递减
- 剪枝 + 量化的质量不一定更好——量化已经在利用参数冗余，剪枝再去掉一部分，两者效果可能叠加而非互补

"剪枝 → 量化"的流水线可能更适合未来的稀疏计算硬件（如 NVIDIA 2:4 稀疏）——而不是 2026 年的主流 GPU。

5.5 三个未来剧本

剧本一：FP4 原生硬件终结软件量化时代（概率 45%）

NVIDIA Blackwell 的 FP4 原生支持改变了游戏规则。如果未来 3-5 年内所有新 GPU 都原生支持 FP4，当前的软件量化方法（GPTQ, AWQ, GGUF）的重要性将大幅下降。"量化"将从软件层的花样变成一个硬件开关。TensorRT-LLM 已经在适配，PyTorch 的 torchao 也在跟进。这个趋势几乎是不可逆的——硬件决定效率上限，软件只能在硬件没跟上时作为权宜之计。

剧本二：蒸馏 + 小模型取代量化大模型成为主流（概率 35%）

如果 DeepSeek-R1 的蒸馏路线被证明在更多领域有效，行业可能从"压缩大模型"转向"训练更好的小模型"。蒸馏 7B 模型 + 4-bit 量化 = 4GB，可以在笔记本上跑，特定领域能力接近 70B 的 80-90%。对于很多应用场景，"90% 的能力 + 1/10 的成本"是更好的商业选择。

这个剧本的变数在于蒸馏的"天花板"——蒸馏模型在通用知识广度上还是不如大模型。但如果未来 2-3 年蒸馏技术能打破这个天花板，整个模型部署的格局将彻底改变。

剧本三：量化精度和模型规模同步增长，压缩沦为"可选优化"（概率 20%）

如果模型规模继续增长（万亿参数级别），但硬件能力（显存、带宽、算力）也同步增长——Blackwell B200 的显存已经达到 192GB——对部分用户来说，压缩的必要性可能降低。不是不需要压缩，而是"跑得起"的门槛在降低。

但即使是这个剧本，量化的部署常规性也不会消失——成本优势不可能被硬件的线性增长抹平。只是压缩的"紧迫性"可能从"不得不做"变成"做了更好"。

5.6 第一性原理追问

模型压缩的终极问题是：**模型的每个参数都有用吗？**

如果答案是"是"，压缩就是对精度的系统性妥协——我们接受损失来换取效率。

如果答案是"否"，压缩就是冗余挖掘——我们只是在去掉模型训练中不可避免产生的浪费。

2026 年的证据指向后者。4-bit 量化几乎无损，SparseGPT 可以移除 50% 的参数而不显著影响质量。DeepSeek-R1-Distill-Qwen-7B 在数学推理上超越 GPT-4o mini——大模型的参数中存储了大量"重复的知识"。

但这个冗余本身可能不是设计的缺陷，而是**涌现的学习特征**——就像人脑的神经元连接也是过度冗余的，冗余提供了容错性和泛化的基础。完全去掉冗余的模型可能失去"在分布外任务上涌现新能力"的潜力。

这也意味着**模型压缩的下限不是"精度能到多少"，而是"冗余的下限在哪里"**——这是一个至今没有答案的问题。

六、信息来源

核心论文

论文	arXiv ID	访问时间
GPTQ	arxiv.org/abs/2210.17323	2026-04-30
AWQ	arxiv.org/abs/2306.00978	2026-04-30
SmoothQuant	arxiv.org/abs/2211.10438	2026-04-30
SpQR	arxiv.org/abs/2306.03078	2026-04-30
SparseGPT	arxiv.org/abs/2301.00774	2026-04-30
Wanda	arxiv.org/abs/2306.11695	2026-04-30
AQLM	arxiv.org/abs/2401.06118	2026-04-30
QuIP#	arxiv.org/abs/2402.04396	2026-04-30
LLM-QAT	arxiv.org/abs/2305.17844	2026-04-30
Knowledge Distillation (Hinton)	arxiv.org/abs/1503.02531	2026-04-30
Lottery Ticket Hypothesis	arxiv.org/abs/1803.03635	2019-04-30
DeepSeek-R1	arxiv.org/abs/2501.12948	2026-04-30
Phi-1	arxiv.org/abs/2306.11644	2026-04-30
LLM Compression Survey	arxiv.org/abs/2312.07629	2026-04-30

产品与框架来源

来源	URL	访问时间
llama.cpp GitHub	github.com/ggerganov/llama.cpp	2026-04-30
vLLM 量化文档	docs.vllm.ai/en/latest/features/quantization.html	2026-04-30
TensorRT-LLM 量化文档	nvidia.github.io/TensorRT-LLM	2026-04-30
bitsandbytes 文档	huggingface.co/docs/bitsandbytes	2026-04-30
AutoGPTQ GitHub	github.com/AutoGPTQ/AutoGPTQ	2026-04-30
AutoAWQ GitHub	github.com/casper-hansen/AutoAWQ	2026-04-30
ExLlamaV2 GitHub	github.com/turboderp/exllamav2	2026-04-30
MLX GitHub	github.com/ml-explore/mlx	2026-04-30
AI 模型压缩综述	arxiv.org/abs/2312.07629	2026-04-30
SGLang GitHub	github.com/sgl-project/sglang	2026-04-30

社区与其他来源

来源	URL	访问时间
Reddit r/LocalLLaMA	reddit.com/r/LocalLLaMA	2026-04-30
LM Studio 博客	lmstudio.ai/blog	2026-04-30
Hugging Face Transformers 量化文档	huggingface.co/docs/transformers/quantization	2026-04-30

本文是横纵分析系列的第 15 篇报告。研究方法论参考了由数字生命卡兹克（Khazix）提出的横纵分析法，融合语言学历时-共时分析、社会科学纵向-横截面研究设计、以及竞争战略分析的核心思想。